

Lab 01 - Workflow business case

AI Agents for Business | TGS-2023018987 | v1.0

Objective

Select a bounded business workflow and calculate net capacity released.

Alignment: K1, B02, B09, B10. Scheduled hands-on time: 30 minutes, with trainer-led interpretation alongside the slides.

Files and prerequisites

- `processes.csv`

Use Python 3.10 or newer and an organisation-approved LLM chat interface. The core local run requires no API key. Model outputs vary: assess evidence and behaviour, not matching prose. All business records are synthetic.

Detailed procedure

1. Open `processes.csv` and identify `volume_per_month`, `manual_minutes`, `review_minutes` and `platform_cost`. All rows describe a fictional SME called Merlion Business Services.
2. Run `python3 run.py`. Open `output.json` and independently recompute `support_triage`: 1000 times (8 minus 3) divided by 60 equals 83.33 hours per month.
3. Use `PROMPTS.md` prompt 1 with the CSV. Require the model to nominate one workflow, its owner, excluded actions and evidence needed before a pilot.
4. Compare the model recommendation with the deterministic calculation. Challenge any assumption that released capacity automatically becomes cash savings.
5. Use prompt 2 to add an exception rate of 20 percent. Recalculate the impact of two additional review minutes on those cases.
6. Save your own decision as `learner-output.json` using `output-template.json`. Record source row IDs, assumptions and one reason to reject or delay the pilot.

Acceptance checks

- Support baseline releases 83.33 hours before additional overhead.
- The proposed pilot includes an owner and an explicit action boundary.
- A sensitivity result and quality floor accompany the financial claim.

Run `python verify.py` after `run.py` (use `python3` if that is your interpreter command). The script verifies deterministic mechanics. Separately complete the evidence-based review of your own LLM output; no script claims an LLM task passed unless its output was actually inspected.

Troubleshooting

- Missing package: confirm the virtual environment is active, then install this folder's `requirements.txt`.
- File not found: open a terminal in this exact lab folder; the script resolves input paths relative to itself.
- Invalid JSON: remove Markdown fences and save a single JSON value; keep the original response for diagnosis.
- Unreliable model answer: reduce the task, include source IDs, inspect each claim and escalate missing evidence.

- Training results differ: record Python/package versions and seed; compare trends and limitations rather than claiming identical scores.

Evidence and cleanup

Keep output.json, learner-output.json and relevant screenshots or logs in your learner submission folder. Stop after verification; do not connect the exercise to real payment, email, HR or production systems. Local generated outputs can be archived after assessment; keep source data and prompts unchanged.